



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2233 — PROGRAMACIÓN AVANZADA 2026-2

Forma B

Interrogación 1

10 de Septiembre 2026

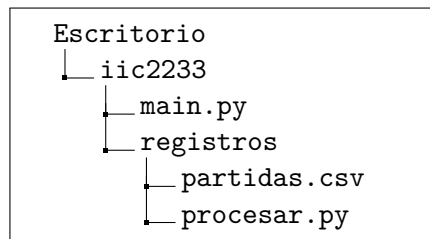
Instrucciones

- La evaluación consta de 3 preguntas de desarrollo.
- Debes completar con tus datos personales e indicar la forma en cada hoja de respuesta utilizada. Luego debes entregar de vuelta **todas** las hojas de respuesta de la prueba, aunque estas se encuentren en blanco.
- Solo podrás retirarte de la sala una vez hayan transcurrido 30 minutos desde que inició de la evaluación.
- Durante la evaluación se realizarán 2 rondas de preguntas. Estas preguntas únicamente serán respondidas por los profesores del ramo.
- En caso de que sea necesario, podrás solicitar hojas blancas para apoyar al desarrollo de la prueba. Para esto levanta la mano y espera que un ayudante se acerque a tu puesto.

Pregunta 1. [30 % nota final]

Parte A. (2 puntos)

Benjamín tiene la siguiente estructura dentro de su Escritorio:



El archivo `procesar.py` contiene la línea:

```
open("partidas.csv")
```

Con la terminal ubicada en `Escritorio`, Benjamín ejecuta:

```
python iic2233/registros/procesar.py
```

y obtiene el siguiente error:

```
FileNotFoundError: [Errno 2] No such file or directory: 'partidas.csv'
```

- A.1) Explica en una o dos líneas por qué ocurre el error, si el archivo `partidas.csv` sí existe.
- A.2) Escribe una secuencia de comandos que permita a Benjamín ejecutar correctamente `procesar.py`, sin que se produzca el error anterior.

Parte B. (2 puntos)

Un ayudante debía procesar un archivo de **1 GB** que contiene mediciones de temperatura registradas por un sensor. Cada línea del archivo almacena una única temperatura, expresada como un número decimal.

Para procesar el archivo, escribió dos versiones de un programa: la primera utilizó `archivo.readlines()` y almacenó todas las líneas antes de procesarlas; la segunda obtuvo las líneas una a una mediante un generador. Ambas versiones realizaron el mismo procesamiento sobre todas las mediciones, pero la segunda utilizó considerablemente menos memoria.

- B.1) Explica en máximo 3 líneas por qué la segunda versión utilizó menos memoria, aunque eventualmente ambas procesaron todas las líneas del archivo.
- B.2) Luego, su compañera quiere aplicar la misma idea en otro programa. En este caso, el programa debe leer las temperaturas del archivo y mostrarlas ordenadas de menor a mayor según su valor numérico.

Considera que el programa ya se encarga de convertir correctamente cada temperatura desde `str` a `float` antes de realizar el ordenamiento.

¿Puede mantenerse de la misma forma la ventaja de memoria obtenida al procesar una línea a la vez? Justifica tu respuesta en un máximo de 3 líneas.

Parte C. (2 puntos)

Considera las siguientes situaciones relacionadas con estructuras de datos en Python.

C.1) Se ejecuta el siguiente código:

```
from collections import namedtuple

Libro = namedtuple("Libro", ["titulo", "autor", "paginas"])

libro_1 = Libro(
    titulo="La metamorfosis",
    autor="Franz Kafka",
    paginas=96
)
```

Luego se intenta ejecutar:

```
libro_1[2] = 450
```

Explica en máximo 2 líneas si esta expresión funciona correctamente o genera un error, y por qué.

C.2) En otro programa de Python que gestiona una biblioteca, se definen dos estructuras de datos de tipo `set`: `prestados`, que almacena los códigos de los libros que fueron prestados, y `devueltos`, que almacena los códigos de los libros que ya fueron devueltos.

Escribe una expresión que entregue los códigos de los libros que aparecen **tanto en prestados como en devueltos**.

C.3) En el mismo programa del inciso (C.2) se intenta ejecutar la siguiente expresión:

```
prestados[:2]
```

Explica en máximo 2 líneas si esta expresión funciona correctamente o genera un error, y por qué.

Pregunta 2. [30 % nota final]

El Bingo es un clásico juego de azar en el cual cada jugador tiene un cartón con 15 números del 1 al 100. El director del juego saca números aleatorios de una tómbola y anuncia el número en voz alta. Cada jugador que tenga el número anunciado en su cartón lo marca con un plumón. Cuando alguien completa todo su cartón, grita bingo y gana un premio.

En el bingo moderno existe un sistema que registra los números de cada jugador para poder manejar el juego de manera eficiente. La siguiente clase registra los números de cartón de los jugadores:

```
class Registro:
    def __init__(self):
        self.registro = []

    def registrar_numeros(self, id_persona: str, carton: tuple) -> None:
        self.registro.append([id_persona, carton])

    def buscar_numeros(self, id_persona: str) -> list:
        cartones = []
        for datos in self.registro:
            if datos[0] == id_persona:
                cartones.append(datos[1])
        return cartones
```

El problema de `Registro` es que no es eficiente para buscar los números de cada jugador.

- A) **(4 puntos)** Reescribe la clase usando las estructuras de datos aprendidas en el curso de manera de que el registro permita buscar de manera eficiente todos los cartones registrados a nombre de una misma persona. Utiliza pseudocódigo.
- B) **(2 puntos)** Explica las propiedades de la o las estructuras de datos usadas que permiten que el nuevo código sea más eficiente.

Pregunta 3. Sistema TAG [40 % nota final]

Una concesionaria utiliza un sistema TAG para calcular y registrar los pagos de los vehículos que atraviesan sus pódicos. Cada vehículo posee una patente y paga un monto calculado a partir de la tarifa base del pódico. Los automóviles pagan la tarifa base que es una constante del sistema, los camiones pagan el doble y las motocicletas pagan la mitad. Para simplificar, todas las tarifas base serán números enteros positivos y pares, por lo que los montos pagados también serán números enteros positivos.

Existe un módulo que contiene las clases `SistemaTAG` y `RegistroPago`. Ambas ya se encuentran implementadas y no deben modificarse. Cada objeto `RegistroPago` posee los atributos `patente: str` y `monto: int`.

Puedes utilizar las siguientes operaciones de `SistemaTAG`:

- `registrar_vehiculo:`

Recibe un vehículo y lo incorpora al sistema. No retorna ningún valor.

- `buscar_vehiculo:`

Recibe una patente y retorna el vehículo asociado. Si la patente no se encuentra registrada, levanta `KeyError`.

- `registrar_pago:`

Recibe un vehículo y la tarifa base del pódico. Calcula el monto mediante el método `calcular_peaje` del vehículo recibido. Si la tarifa base es menor o igual a cero, levanta `ValueError` y no registra el pago. En caso contrario, conserva la patente y el monto pagado en un nuevo objeto `RegistroPago`, lo registra y lo retorna.

- `obtener_registros:`

Retorna una lista de objetos `RegistroPago` en el mismo orden en que fueron agregados mediante `registrar_pago`. El primer elemento corresponde al primer pago registrado, el segundo al segundo pago registrado, y así sucesivamente. Los registros no se ordenan por patente ni por monto.

Un mismo vehículo puede realizar varios pagos. Todos estos pagos deben conservarse, incluidos aquellos asociados a una misma patente, y mantenerse en el orden en que fueron registrados.

Los pódicos comunican cada cobro mediante un texto con el formato `PATENTE;TARIFA_BASE`. Por ejemplo, `ABCD12;1500` representa un cruce de la patente `ABCD12` con tarifa base 1500. Una lectura es inválida si no contiene exactamente una patente no vacía y una tarifa base que pueda convertirse a un entero positivo. Estos casos deben manejarse mediante `ValueError`. Si la patente no está registrada, debe manejarse el `KeyError` levantado por `buscar_vehiculo`. Ante cualquiera de estos errores, el programa debe mostrar un mensaje en consola, no registrar el pago y continuar funcionando.

Consideraciones

- En el manejo de errores se deben distinguir las situaciones asociadas a `ValueError` y `KeyError`. Una solución que capture únicamente `Exception`, sin considerar las excepciones específicas descritas en el enunciado, no se considerará correcta.

A partir de lo anterior, responde las siguientes preguntas:

- A) **(2 puntos)** Construye un diagrama de clases UML para modelar el problema. El diagrama debe incluir **SistemaTAG**, **RegistroPago**, y todas las otras clases necesarias para el correcto modelamiento del problema.

Justifica brevemente cómo tu modelación permite conservar varios pagos asociados a una misma patente y mantenerlos en el orden en que fueron registrados.

- B) **(2.5 puntos)** Escribe en pseudocódigo la siguiente función:

```
resumir_pagos(sistema: SistemaTAG, patente: str)
```

La función debe comprobar que la patente corresponde a un vehículo registrado en el sistema y calcular la cantidad de pagos y el monto total pagado por ese vehículo.

Debe retornar ambos resultados. Si el vehículo está registrado, pero todavía no posee pagos, debe retornar una cantidad y un monto total iguales a cero.

Si la patente no corresponde a un vehículo registrado, debe mostrar un mensaje en consola, continuar la ejecución del programa y retornar **None**.

- C) **(1.5 puntos)** Escribe en pseudocódigo la siguiente función:

```
consultar_montos(sistema: SistemaTAG, patente: str)
```

La consulta debe entregar, uno por uno, los montos de todos los pagos asociados a la patente recibida, respetando el orden en que los registros fueron almacenados.

Si existen varios registros asociados a la patente, debe entregar el monto de cada uno de ellos, incluso cuando algunos montos sean iguales. Si no existen pagos asociados a la patente, no debe entregar ningún resultado.

Para probarla, considera los siguientes registros:

Patente	Monto pagado
AA1111	1500
BB2222	2000
AA1111	2001
CC3333	2500
AA1111	2001

Si la patente recibida es AA1111, la función debe entregar, en este orden:

1500, 2001 y 2001.

Nombre completo: _____

RUN: _____ Sección: ____ Forma: ____ N° lista: _____ Página: ____ de ____

IIC2233 - Pregunta 1

Nombre completo: _____

RUN: _____ Sección: ____ Forma: ____ N° lista: _____ Página: ____ de ____

IIC2233 - Pregunta 2

Nombre completo: _____

RUN: _____ Sección: ____ Forma: ____ N° lista: _____ Página: ____ de ____

IIC2233 - Pregunta 3